极客时间

# Hooks 事件驱动自动化

工程思想：反馈驱动循环

第 2 周 · 第 5 节

讲师：黄佳

极客时间黄佳老师出品

# V1 回顾：第 1 周产出清单

极客时间

**AGENTS.md（Memory）** — 项目规范、技术栈、内容风格、红线

**3 个 Agent** — collector（采集）、analyzer（分析）、organizer（整理）

**2 个 Skill** — github-trending（7 步采集）、tech-summary（4 步分析）

**知识数据** — knowledge/raw/*.json + 分析结果 JSON

**V1 的工作方式（编码自动化）：**

- OpenCode 生成代码骨架 → 但每步都要你驱动对话触发
- 采集、分析、整理、质量检查 → 全靠你手动发起

**V1 → V2 升级目标：**

- 从“人驱动对话'到“代码自己跑 + 自动校验”
- 本节：Hooks 实现自动校验 | 后续：MCP + CI/CD 实现全自动

极客时间黄佳老师出品

# V1 的三个痛点——为什么需要自动化?

极客时间

**痛点 1：人驱动** — 代码生成靠 OpenCode，但每步还要你发起对话触发

- 忘了触发 → 数据断更；依赖人驱动 → 效率低、容易遗漏

**痛点 2：无质量检查** — 产出直接入库，没有质量门卫

- JSON 格式错误 → 下游解析失败
- 必填字段缺失 → 信息不完整
- 摘要空洞无物 → 知识库价值归零

**痛点 3：无反馈循环** — 出了问题不知道，知道了也不自动修

- 开环系统：输入 → 处理 → 输出 → （无人检查）
- 没有“感知→判断→纠正”的自动回路

**核心问题：如何让 Agent 自己检查、自己纠错?**

极客时间

极客时间

工程思想

# 反馈驱动循环
# Gather → Act → Verify → Repeat

Agent 不是一次性脚本，它的本质是持续运行的反馈循环
Hooks 就是循环中的“验证”节点

# 什么是 Hook？事件触发的自动化代码

极客时间

**Hook = 在特定时机自动执行的代码**

- 不需要你手动调用 — 事件发生时自动触发

**类比：常见的 Hook：**

- Git Hooks — commit 前自动跑 lint（pre-commit）
- CI/CD Pipeline — push 后自动跑测试（GitHub Actions）
- React Hooks — 组件渲染后自动执行副作用（useEffect）

**Agent 场景的 Hook：**

- Agent 要执行工具前 → 自动检查权限（tool.execute.before）
- Agent 写完文件后 → 自动校验格式（tool.execute.after）
- Agent 完成任务前 → 自动做最终质量检查（Stop）

**本质：把人工检查变成自动检查**

极客时间黄佳老师出品

# OpenCode Plugin Hook 体系 — 5 种 Hook 类型

极客时间

| Hook 类型 | 触发时机 | 典型用途 | 返回值控制 |
| --- | --- | --- | --- |
| tool.execute.before | 工具执行前 | 权限拦截、参数检查 | 可阻止执行 |
| tool.execute.after | 工具执行后 | 格式校验、日志记录 | 可触发修正 |
| Notification | Agent 发送通知时 | 消息推送、状态记录 | 仅观察 |
| UserPromptSubmit | 用户提交 prompt 时 | 输入预处理、安全审查 | 可修改输入 |
| Stop | Agent 停止前 | 最终质量门、完整性检查 | 可阻止停止 |

# OpenCode Plugin Hook 体系 — 真实 API

极客时间

**插件位置：** .opencode/plugins/*.ts（TypeScript 文件，自动加载）
**包名：** @opencode-ai/plugin（注意不是 @opencode/plugin）

**核心 Hook 事件（实际支持的）：**

- tool.execute.before — 工具执行前触发（可拦截）
- tool.execute.after — 工具执行后触发（可校验产出）
- chat.message — 收到消息时触发
- command.execute.before — 命令执行前触发
- shell.env — 注入环境变量

**Plugin 签名（真实 API）：**

```
export const MyHook: Plugin = async ({ $ }) => {
  return { "tool.execute.after": async (input) => { ... } }
}
```

- → 不是 definePlugin()，是直接导出 Plugin 类型的函数

极客时间

# Hook 详解 — tool.execute.before：权限门卫

极客时间

**触发时机：** Agent 准备调用工具时，在实际执行前触发

- **用途：** 拦截危险操作、检查权限、修改参数

**实战场景：**

- Agent 想删除文件 → before Hook 检查路径
- → 如果是 knowledge/ 目录 → 拦截，返回警告
- → 如果是 /tmp/ 临时文件 → 放行
-

**类比：** 大楼门卫

- 快递员要进大楼（Agent 要执行工具）
- 门卫检查工牌（before Hook 检查权限）
- 有工牌 → 放行 | 没工牌 → 拦截

**本节课重点不在 before，而在 after**

- 因为我们的场景是'写完文件后校验'，不是'写之前拦截'

极客时间黄佳老师出品

# Hook 详解 — tool.execute.before：权限门卫

极客时间

opencode plugin hook | TypeScript

```typescript
// tool.execute.before Hook：拦截危险命令
// 文件：.opencode/plugins/validate.ts

export function onToolExecuteBefore(event: ToolUseEvent)
{
  const { tool, input } = event;

  // 仅拦截 bash 工具
  if (tool !== 'bash') return { decision: 'allow' };

  const command = input.command || '';

  // 定义危险命令黑名单
  const dangerous = [
    /rm\s+-rf\s+\//, // rm -rf /
    /DROP\s+TABLE/i,   // SQL 删表
    /:(){ :|:& };:/,  // Fork 炸弹
    /mkfs\./,         // 格式化磁盘
  ];
```

opencode plugin hook | TypeScript

```typescript
  for (const pattern of dangerous) {
    if (pattern.test(command)) {
      return {
        decision: 'deny',
        reason: `Blocked: ${command.slice(0, 50)}...`
      };
    }
  }
  return { decision: 'allow' };
}
```

极客时间黄佳老师出品

# Hook 详解 — tool.execute.after：质量检查员

极客时间

opencode plugin hook | TypeScript

```typescript
import type { Plugin } from "@opencode-ai/plugin"

export const ValidateHook: Plugin = async ({ $ }) => {
  return {
    "tool.execute.after": async (input) => {
      // 只在 write/edit 工具执行后触发
      const tool = input.tool?.toLowerCase() ?? ""
      if (tool !== "write" && tool !== "edit") return

      // 获取文件路径
      const filePath = input.args?.file_path
                    ?? input.args?.filePath
      if (!filePath) return

      // 只校验 knowledge/articles/ 下的 JSON
      if (!filePath.includes("knowledge/articles/")
          || !filePath.endsWith(".json")) return
```

opencode plugin hook | TypeScript

```typescript
      // 调用 Python 校验脚本（Bun Shell API）
      const r = await
        $`python3 hooks/validate_json.py ${filePath}`
          .nothrow()

      if (r.exitCode !== 0)
        console.error("[hook] JSON 校验失败")
      else
        console.log("[hook] JSON 校验通过")
    },
  }
}
export default ValidateHook
```

极客时间黄佳老师出品

# Hook 详解 — Stop：最终质量门

极客时间

opencode plugin hook | TypeScript

```
// Stop Hook：Agent 停止前做最终质检
// 确保所有知识条目都通过了质量校验

export function onAgentStop(event: StopEvent) {
  const { context } = event;

  // 检查本次任务是否涉及知识库操作
  if (!context.task.includes('knowledge')) {
    return { allow: true };
  }

  // 运行 Python 质量校验脚本
  const result = exec(
    'python scripts/validate_json.py knowledge/raw/'
  );

  if (result.exitCode !== 0) {
    return {
      allow: false,  // 阻止 Agent 停止
      feedback: '质量检查未通过，请修正以下问题：\n'
              + result.stderr
    };
  }

  return { allow: true }; // 通过，允许停止
}
```

# 对比：OpenCode Hook vs Claude Code Hook

极客时间

## OpenCode Plugin Hook

配置方式：TypeScript 代码
文件位置：.opencode/plugins/*.ts
语言：TypeScript + Bun Shell
事件名：tool.execute.before/after
加载：启动时自动发现

优势：代码灵活，可调用任意程序
劣势：需要 TypeScript 基础

## Claude Code Hook

配置方式：JSON 声明式
文件位置：.claude/settings.json
语言：Shell 脚本
事件名：PreToolUse / PostToolUse
加载：读取 JSON 配置

优势：声明式，不需要写代码
劣势：只能跑 Shell 脚本

极客时间

# 实战：Python 质量校验脚本

独立于 OpenCode 的校验工具 — 可被 Hook 调用，也可独立运行

极客时间黄佳老师出品

# 代码展示 — validate_json.py（格式校验脚本）

极客时间

scripts/validate_json.py | Python

```python
import json, sys
from pathlib import Path

REQUIRED_FIELDS = ['source', 'collected_at', 'items']
ITEM_REQUIRED = ['name', 'url', 'summary', 'relevance_score']
ID_PATTERN = r'^[a-z0-9-]+$'  # ID 格式：小写字母+数字+连字符

def validate_file(path: Path) -> list[str]:
    errors = []
    try:
        data = json.loads(path.read_text('utf-8'))
    except json.JSONDecodeError as e:
        return [f'JSON 语法错误: {e}']

    # 顶层必填字段
    for f in REQUIRED_FIELDS:
        if f not in data:
            errors.append(f'缺少顶层字段: {f}')
```

scripts/validate_json.py | Python

```python
    # 逐条检查 items
    for i, item in enumerate(data.get('items', [])):
        for f in ITEM_REQUIRED:
            if f not in item:
                errors.append(f'items[{i}] 缺少: {f}')
        # 评分范围检查
        score = item.get('relevance_score', 0)
        if not (1 <= score <= 10):
            errors.append(f'items[{i}] 评分超范围: {score}')
        # 摘要长度检查
        summary = item.get('summary', '')
        if len(summary) > 100:
            errors.append(f'items[{i}] 摘要过长: {len(summary)}字')
    return errors
```

# validate_json.py 运行效果

极客时间

validate_json.py

```
$ python3 hooks/validate_json.py knowledge/articles/test-good.json

==================================================
JSON 格式校验结果
==================================================

[PASS] 所有文件校验通过

总计: 1 文件, 1 通过, 0 失败
```

validate_json.py

```
$ python3 hooks/validate_json.py knowledge/articles/test-bad.json

==================================================
JSON 格式校验结果
==================================================

[FAIL] knowledge/articles/test-bad.json
  - 缺少必填字段: source_url
  - 缺少必填字段: summary
  - 缺少必填字段: tags

总计: 1 文件, 0 通过, 1 失败
```

# 代码展示 — check_quality.py（5 维度质量评分）

极客时间

scripts/check_quality.py | Python

```python
# 5 维度质量评分
WEIGHTS = {
    'summary_quality': 0.30,  # 摘要质量 30%
    'tech_depth':      0.25,  # 技术深度 25%
    'format_compliance': 0.20, # 格式规范 20%
    'tag_quality':     0.15,  # 标签质量 15%
    'hollow_penalty':  0.10,  # 空洞用词 10%（扣分项）
}

def score_summary(text: str) -> float:
    """摘要质量：有具体信息得高分，空洞泛泛得低分"""
    score = 1.0
    if len(text) < 10: score -= 0.5    # 太短
    if len(text) > 80: score -= 0.3    # 太长
    if not any(c.isdigit() for c in text):
        score -= 0.2  # 无数字 = 可能缺具体信息
    return max(0, score)
```

scripts/check_quality.py | Python

```python
def score_hollow(text: str) -> float:
    """空洞用词检测（扣分项：分数越高越好）"""
    BLACKLIST = [
        '非常强大', '令人印象深刻', '颠覆性',
        '革命性', '前所未有', '独一无二',
        '业界领先', '遥遥领先', '无与伦比',
    ]
    hits = sum(1 for w in BLACKLIST if w in text)
    return max(0, 1.0 - hits * 0.3)  # 每命中一个扣 0.3

def overall_score(item: dict) -> float:
    scores = {
        'summary_quality':
score_summary(item.get('summary','')),
        'tech_depth': ...,  # 类似逻辑
        'format_compliance': ...,
        'tag_quality': ...,
        'hollow_penalty': score_hollow(str(item)),
    }
    return sum(scores[k]*WEIGHTS[k] for k in
WEIGHTS)
```

# check_quality.py 运行效果

check_quality.py

```
$ python3 hooks/check_quality.py knowledge/articles/test-bad.json

文件: knowledge/articles/test-bad.json

  摘要质量    [                                                  ]  0.0/25  无摘要
  技术深度    [▇▇▇▇▇▇▇▇▇▇▇▇                                      ] 12.5/25  文章评分 5/10 → 12.5/25.0
  格式规范    [▇▇▇▇▇▇▇▇▇▇▇▇▇▇▇▇                                  ] 12.0/20  缺失: 缺少 source_url, 缺少时间戳
  标签精度    [                                                  ]  0.0/15  无标签
  空洞词检测   [▇▇▇▇▇▇▇▇▇▇▇▇▇▇▇▇
```

# 空洞用词黑名单 — AI 最爱用的空话

极客时间

**为什么要检测空洞用词?**

- AI 生成的文本天然倾向于使用模糊的、积极的、无信息量的表达
- 知识库要的是事实和数据，不是营销文案

**黑名单词汇（示例）：**

- 非常强大 → 改为：支持 50+ 平台的编排调度
- 令人印象深刻 → 改为：推理速度比 GPT-4 快 3 倍
- 颠覆性的创新 → 改为：首次实现端到端多模态推理
- 革命性 → 改为：将训练成本降低了 90%
- 业界领先/ 遥遥领先'→ 改为：GitHub Star 数超过 10 万

**检测规则：**

- 每命中 1 个黑名单词 → 扣 0.3 分（满分 1.0）
- 命中 3 个以上 → 该条目直接标记为“需要重写”

**原则：用事实替代形容词，用数字替代感叹号**

极客时间

# Hook 配置实战 — .opencode/plugins/ + 独立脚本

极客时间

.opencode/plugins/knowledge-guard.ts | TypeScript + Shell

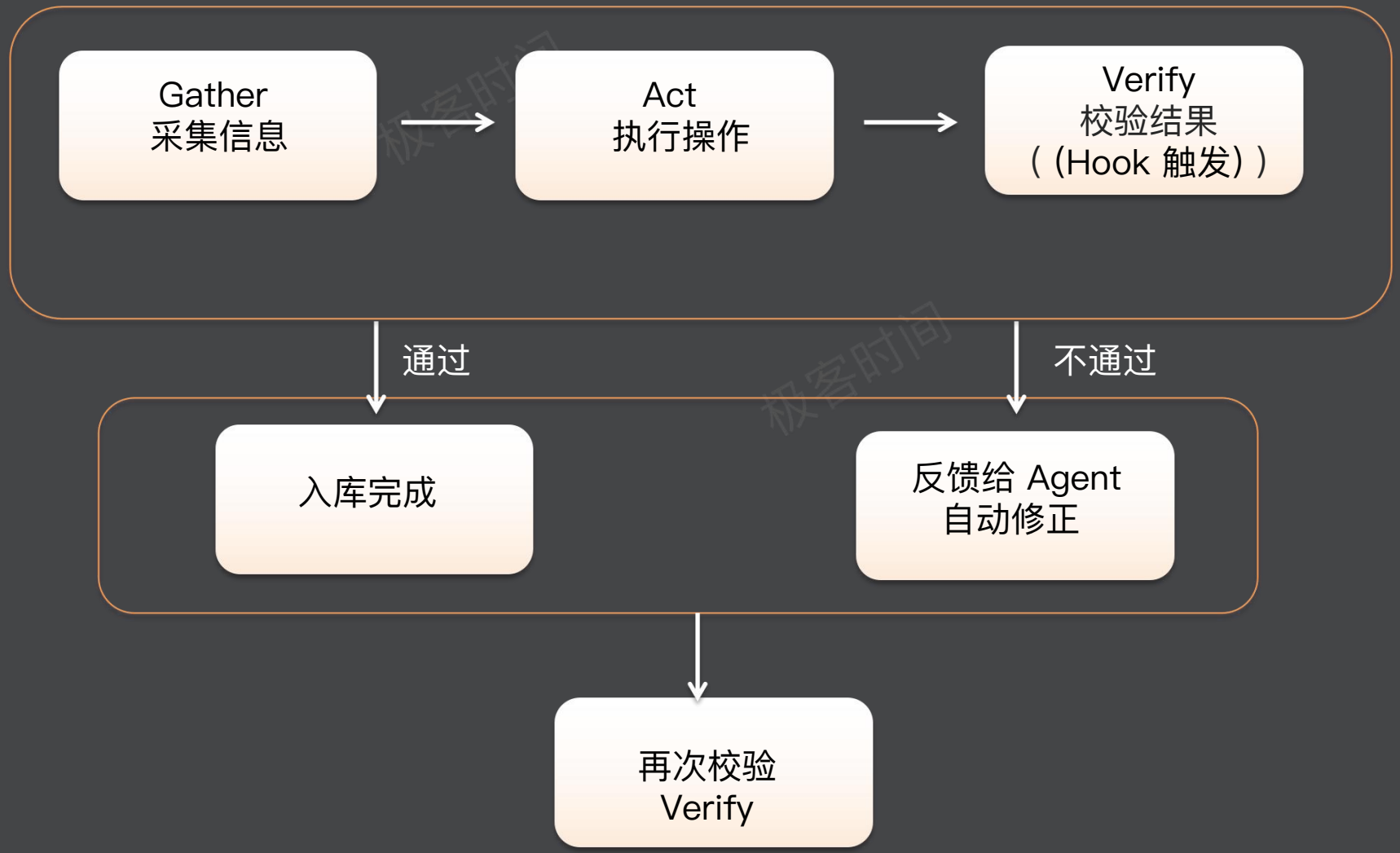
```
// — 方式 1：在 .opencode/plugins/ 中编写 TypeScript 插件 —
// 文件：.opencode/plugins/knowledge-guard.ts

import { definePlugin } from '@opencode/plugin';

export default definePlugin({
  name: 'knowledge-guard',
  hooks: {
    'tool.execute.before': preToolCheck,
    'tool.execute.after':  postToolValidate,
    'stop':                stopQualityGate,
  },
});
```

.opencode/plugins/knowledge-guard.ts | TypeScript + Shell

```
// —— 方式 2：独立运行 Python 校验脚本
————————————————————————————————

// 不依赖 OpenCode，可在任何环境使用

$ python scripts/validate_json.py knowledge/raw/
  # → 检查所有 JSON 文件，报告错误

$ python scripts/check_quality.py knowledge/raw/*.json
  # → 5 维度评分，输出不达标条目
```

极客时间黄佳老师出品

# 反馈驱动循环 — 完整运行图




极客时间黄佳老师出品

# 对比：有 Hook vs 无 Hook

极客时间

## 无 Hook = 开环系统

Agent 执行 → 直接输出 → 结束

问题发现：靠人工检查（经常忘记）
修正方式：手动重新跑
质量保证：无

类比：
工厂没有质检 → 次品流到客户
写代码没有测试 → bug 上线
开车没有仪表盘 → 不知道速度

结果：产出质量不稳定
好的时候很好，差的时候很差
取决于人的注意力和记忆力

## 有 Hook = 闭环系统

Agent 执行 → Hook 校验 → 反馈 → 修正 → 再校验

问题发现：自动（Hook 触发）
修正方式：Agent 自动修正
质量保证：持续

类比：
工厂有质检 → 次品被拦截
有 CI/CD → 测试不过不上线
有仪表盘 → 超速自动报警

结果：产出质量稳定一致
不依赖人的注意力
7x24 自动运行，质量有底线

# 本节作业

极客时间

## 任务 1：编写 JSON 格式校验脚本（validate_json.py）

参考课上展示的代码，完成完整的 validate_json.py。要求：(1) 检查 JSON 语法合法性 (2) 检查顶层必填字段 (3) 逐条检查 items 的必填字段 (4) 评分范围 1–10 (5) 摘要长度 ≤100 字。在命令行运行，对 knowledge/raw/ 中的文件做校验。

## 任务 2：编写质量评分脚本（check_quality.py）

实现 5 维度质量评分：摘要质量 30% + 技术深度 25% + 格式规范 20% + 标签质量 15% + 空洞用词 10%。重点实现空洞用词黑名单检测。对 V1 产出的知识条目做评分，找出不达标的条目。

## 任务 3：在 OpenCode 中配置 Hook 并观察反馈循环

在 .opencode/plugins/ 中编写 tool.execute.after Hook 插件，调用 validate_json.py。然后故意让 Agent 写一个有问题的 JSON（缺字段或空洞用词），观察“ Hook 触发 → 反馈 → Agent 修正 → 再次校验”的完整循环过程。截图记录反馈循环的每一步。

# 工程思想总结卡：反馈驱动循环

极客时间

**一句话** — Agent 系统的可靠性不来自执行能力，而来自反馈循环

**为什么重要：**

- 没有反馈循环的系统是开环的 — 错误会累积、扩散、不可控
- 有反馈循环的系统是闭环的 — 错误会被检测、修正、收敛

**本节验证：**

- tool.execute.before — 执行前拦截危险操作（预防错误）
- ool.execute.after — 执行后校验产出质量（发现错误）
- Stop — 停止前最终质检（兜底检查）
- Python 脚本 — 独立可用的校验工具（多场景复用）

**通用原则（超越 AI 编程）：**

- 任何工程系统要可靠运行，都需要反馈循环
- 代码有测试、工厂有质检、飞机有仪表 — 同一个原理
- 从'做了'到'做对了' — 差的就是一个 Verify 节点

极客时间

极客时间

# 本节小结

极客时间

Hooks = 工具执行前后的自动化钩子

before hook：拦截 / 校验 / 修改（守门员）

after hook：通知 / 记录 / 触发下游（传令兵）

V1 痛点：人驱动对话 → V2 目标：代码自己跑

极客时间

**下一节：第 6 节 · MCP 外部工具接入 + 工程思想：标准化接口**